



Editorial

Tasks, test data and solutions were prepared by: Vito Anić, Gabrijel Jambrošić, Josip Klepec, Martina Licul, and Patrick Pavić.

Implementation examples are given in attached source code files.

Task 7Krokods

Prepared by: Martina Licul

Necessary skills: if statement, for loop

In the first subtask, Paula has no crocodile cards, so we only need to compute the score as described in the task statement.

In the second subtask, Paula has one crocodile card. We can try replacing the card with each of the letters **d**, **k**, **o**, and **r**, calculate the score for each, and then output the maximum among the scores.

In the third subtask, Paula can have up to 10 crocodile cards. We want to try all possible combinations of replacing crocodile cards, i. e. we want to find all combinations of numbers **D**, **K**, **O** and **R** ($0 \leq d, k, o, r \leq m$) such that $d + k + o + r \leq m$, where m is the number of crocodile cards. Since m is only up to 10, we can try all combinations with 4 nested loops, each going from 0 to m , and each representing the number of crocodile cards that will be replaced with a certain letter. For each combination, we will check if their sum isn't bigger than m , and then calculate the score. As in the second subtask, we will output the maximum among the scores.

Task Zrinka

Prepared by: Josip Klepec

Necessary skills: dynamic programming

The first two subtask can be solved using greedy approach.

For full points we will use dynamic programming.

Let $dp(i, j, flag)$ represent the solution if we only consider the first i positions of the first array, the first j positions of the second array, and if the largest number is in the first array, then $flag = 0$, otherwise $flag = 1$ (and the largest number is in the second array).

For the transition we need only to consider 4 states: $((i - 1, j, 0), (i - 1, j, 1), (i, j - 1, 0), \text{ and } (i, j - 1, 1))$, so the final complexity is $O(nm)$.

For details check the official code.

Task Bojanje

Prepared by: Gabrijel Jambrošić

Necessary skills: dynamic programming, matrices, probabilities

To solve the first subtask, notice that, in order to not to decrease the number of different colours, Oliver has to choose the same part of the painting in both steps. The probability that this will happen is $\frac{1}{n}$ per iteration, and $(\frac{1}{n})^t$ after t iterations.

The second subtask can be solved by using dynamic programming. The states will be all partitions of the number n (sorted array of frequencies of each of the colours.) because symmetrical cases when we rotate the frequencies of each colour are redundant. Now we go through all combinations of Oliver's choice of two parts and determine in which partition of n it will change into. This is the transition. We will repeat it t times and then sum up all probabilities of partitions that consist of at least k colours.



For solving the last subtask, it is necessary to notice that we are using always the same transition, and that, actually, for each of the elements from the current state is a linear combination of elements from the previous states. Because of that, we can store the states in a mathematical vector and create a transition matrix, and then apply the matrix t times.

Task Mreža

Prepared by: Vito Anić

Necessary skills: binary search, persistent segment tree (alternatively parallel binary search), lowest common ancestor

It is always optimal to repair the roads which have the smallest speed. We can solve the first subtask just with this observation. For every query with using dfs we can find all the edges on the required path. Now we just update the slowest edge until we run out of money. Time complexity $O(nq \log n)$

Let us assume that we want to have the minimal driving speed x on some path. If we want to traverse through some edge i there are three possible outcomes:

- $x \leq v_i$: in this case the road speed is fast enough, we do not need to pay for road upgrade
- $v_i < x \leq s_i$: in this case we need to upgrade the road to pass through it
- $s_i < x$: in this case we cannot pass through this road at all. We will say that the price to pass through this edge is infinite.

Every edge has three possible 'states'. If x is smaller or equal than its speed, we do not to pay for it, if it is between v_i and s_i , we need to pay for it, and if it is larger than s_i we need to pay for it 'infinite' amount of coins. Therefore, in two moments the state of the edge will change: when $x = v_i + 1$ and when $x = s_i + 1$. We will call that moments 'events'. Let us gather events of all edges and sort them.

From this moment two different solution of this problem is possible. We can use parallel binary search to solve all queries at the same time. Every time we would go through all events and update them in segment tree, and at certain moments return the sum of values on the path of some query. The second solution is to use persistent segment tree. We would then for every query binary search the solution individually. In both solution the final time complexity is $O(n \log n^2)$.

Task Vrsta

Prepared by: Vito Anić

Necessary skills: number compression, segment tree

First, let us notice that we are only interested in the height of a student leading the warm-up. We can just track the height and the number of students of that height. We can compress the heights of students because to determine the height of a student who will lead the warm-up, we only need to know how many students are taller and how many are shorter than him. So now the student heights are numbers from 1 to 200 000. We can now go through the order of students entering the gymnasium and will update the segment tree. The i -th leaf of the segment tree will track the current number of students of height i in the gymnasium. The values of other nodes will be the sum of the children's values. If the current number of children is x , then the warm-up will lead $\lfloor \frac{x}{2} \rfloor$ -th child in a sorted line. Let that number be y . Now let's determine the height using the segment tree. Start from the root of the segment tree and use this algorithm until you reach the leaf: If the value of the left child is greater or equal to y , then go to the left child. If the value is smaller than y , then reduce the y by the value of the left child and go to the right child. The leaf index will be the compressed height of the required height. Total time complexity $O(n \log n)$.